

Multi-Agent Framework for Uncertainty-Aware Resource Allocation and Auto-Scaling in Distributed Systems

Harshita Bogineni^{1†}, Sivani R^{1†}, Sivaranjani A^{1*}

¹School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, Tamil Nadu, 632014, India.

*Corresponding author(s). E-mail(s): sivaranjani.a@vit.ac.in;
Contributing authors: harshita.bogineni2023@vitstudent.ac.in;
sivani.r2022@vitstudent.ac.in;

[†]These authors contributed equally to this work.

Abstract

Distributed systems must continually adjust resource supply under volatile and partially observable workloads. Conventional threshold-based or purely reactive autoscalers are attractive because they are simple, but prior surveys and empirical studies report persistent problems with lag, oscillation, and poor handling of non-stationary demand. This paper presents a complete research artifact for uncertainty-aware resource allocation and auto-scaling in distributed systems using the Springer Nature `sn-jnl` format. The framework is implemented as a modular FastAPI microservice stack comprising telemetry simulation, workload generation, lightweight recurrent forecasting, uncertainty-aware risk scoring, multi-agent evaluation, greedy allocation, centralized decision coordination, execution, evaluation, observability, and experiment orchestration. The control policy uses explicit uncertainty, trend, volatility, and multi-agent utility signals rather than raw thresholds alone. The empirical study is grounded in the validated publication suite bundled with the project: 390 successful runs across 13 scenarios, with 30 runs per scenario, 20 timesteps per run, and exported confidence intervals, permutation tests, Holm-Bonferroni corrections, Cliff’s delta effect sizes, and paper-ready artifacts. In the core baseline comparison, the proposed policy matches threshold and reactive baselines on mean RMSE and average utilization while exhibiting substantially richer bidirectional control behavior than threshold control and much lower action churn and cost than the predictive-only baseline. Real-trace experiments based on a normalized Google cluster extract further show that the proposed controller preserves

adaptive behavior while avoiding the fully static hold pattern observed in the threshold policy. The resulting system is best interpreted as a reproducible, publication-grade research prototype: it does not yet claim production superiority, but it closes important gaps in reproducibility, uncertainty integration, and interpretable multi-agent coordination for cloud-native auto-scaling research.

Keywords: distributed systems, auto-scaling, resource allocation, uncertainty-aware control, multi-agent systems, cloud-native systems, reproducibility

1 Introduction

Elastic resource management is a foundational problem in distributed systems because demand varies over time while service operators must simultaneously control cost, latency, and stability. Recent surveys describe auto-scaling as a central enabler of cloud elasticity, yet they also note that many practical controllers remain dominated by threshold or short-horizon reactive logic that is easy to deploy but often brittle under shifting workloads [2, 3, 19]. Empirical work has shown that better metric selection and learning-based automation can sharply reduce quality-of-service violations relative to naive autoscaling rules [1], which motivates the search for controllers that can reason beyond the current utilization sample.

This paper studies that problem through a complete implementation rather than an abstract policy sketch. The project repository contains eleven FastAPI microservices, containerized execution via Docker Compose, Kubernetes deployment manifests, Prometheus and Grafana observability, a deterministic experiment runner, publication-bundle validation scripts, and persistent result exports. The central research question is whether an uncertainty-aware multi-agent control pipeline can deliver meaningfully different scaling behavior than threshold-only or reactive baselines while remaining interpretable and reproducible.

The paper makes three contributions. First, it presents an end-to-end uncertainty-aware scaling framework that integrates forecasting, explicit risk estimation, multi-agent scoring, allocation planning, and execution in a modular service architecture. Second, it grounds every methodological claim in the implemented code and validated publication artifacts, including the 390-run April 16, 2026 publication suite. Third, it situates the framework against recent literature on machine-learning autoscaling, reinforcement-learning resource management, uncertainty-aware scaling, multi-agent resource allocation, and Kubernetes scheduling [4–12, 14, 15, 17–19].

2 Background and Related Work

2.1 ML-Based Autoscaling and Resource Allocation

Auto-scaling surveys consistently distinguish reactive rules from predictive or learning-based controllers and argue that the latter are increasingly necessary for cloud-native systems with bursty, non-stationary demand [2, 3, 19]. Pintye *et al.* show that careful metric selection for machine-learning autoscaling can reduce QoS violations by

roughly 80% while also lowering virtual-machine usage [1]. Broader comparative reviews likewise report that machine-learning and hybrid AI methods outperform classical heuristics on makespan, cost, and energy objectives in cloud resource allocation [4]. These studies support the basic premise of augmenting simple rule systems with richer predictive signals, but they also reveal a reproducibility problem: workloads, metrics, and experimental protocols vary widely across papers [19].

2.2 RL-Based Resource Management

Reinforcement learning has become a major branch of auto-scaling research because it can represent sequential control and delayed reward directly. Surveys of RL-based resource management describe applications to provisioning, placement, scheduling, offloading, and load balancing, particularly in fog and edge systems [5]. Recent microservice and workflow studies instantiate this pattern with concrete algorithms. Nayir *et al.* propose a DQN-based multi-dimensional autoscaler for microservices that jointly adjusts pod count and resource limits [11]. Hu *et al.* combine meta-learning with TD3 in MSARS to accelerate SLO-aware adaptation [12]. Gari *et al.* compare Q-learning and SARSA for workflow autoscaling and find that SARSA converges faster and achieves lower execution time and cost [15]. These methods are powerful, but they typically incur training overhead, depend on reward design, and can be difficult to audit when behavior changes.

2.3 Uncertainty-Aware Scaling

A second branch of the literature explicitly models uncertainty. MagicScaler is particularly relevant because it combines predictive autoscaling with quantified uncertainty using multi-scale Gaussian processes and stochastic optimization, allowing cost-QoS trade-offs to be expressed directly [6]. Chen *et al.* extend uncertainty-awareness into fuzzy-adaptive hierarchical reinforcement learning [7], while Aquatope uses Bayesian models to manage QoS-sensitive serverless workflows under uncertainty [13]. This literature makes two points that motivate our design. First, confidence information materially changes scaling decisions. Second, uncertainty need not imply an opaque controller: it can be encoded in interpretable, bounded signals.

2.4 Multi-Agent Resource Allocation

Distributed resource allocation also has a strong multi-agent lineage. Surveys of distributed algorithms and multi-agent reinforcement learning frame resource allocation as a coordination problem involving multiple decision makers, local observations, and partially aligned objectives [8, 9]. Role-based MARL for cloud-native clusters improves utilization and scheduling latency by coordinating heterogeneous agents with shaped rewards [17], while MARL-game-theoretic optimization improves SLA satisfaction in network slicing scenarios [18]. Our work does not attempt learned multi-agent coordination; instead, it uses an interpretable multi-agent scoring layer that separates performance, cost, and risk perspectives. This makes the system less adaptive than full MARL, but more auditable and easier to reproduce.

2.5 Kubernetes and Cloud-Native Scheduling

Kubernetes remains the de facto execution substrate for cloud-native services, which makes scheduler behavior and autoscaling interfaces highly relevant [10]. Experimental work on Horizontal Pod Autoscaling shows that HPA can improve throughput and latency under some workloads, but often at the cost of higher resource usage and with limited awareness of inter-service dependencies or uncertainty [14]. Recent surveys emphasize that cloud-native autoscaling still lacks holistic controllers that jointly reason about workload prediction, uncertainty, multiple objectives, and reproducibility [19].

2.6 Research Gap and Positioning

The literature reveals four gaps that directly shape our framework. First, many autoscalers still omit explicit uncertainty and risk modeling even when they use predictive components [6, 7, 19]. Second, reproducibility is often weak: results depend on custom traces, undocumented parameterizations, or incomplete artifact release [4, 19]. Third, multi-agent coordination is usually studied through learned MARL formulations, but there is less guidance on interpretable, modular coordination layers that can be inspected service by service [8, 9]. Fourth, risk-aware decision frameworks are comparatively rare despite the operational importance of conservative action under ambiguity [6, 7].

Relative to MagicScaler [6], the present framework uses a lighter recurrent forecaster and a bounded additive risk score rather than Gaussian-process uncertainty and stochastic optimization. Relative to RL-based autoscalers such as DQN, TD3, Q-learning, and SARSA controllers [11, 12, 15], it avoids training overhead and opaque reward shaping, but sacrifices the ability to learn directly from long-horizon environment feedback. Relative to Kubernetes HPA-style control [14], it explicitly models uncertainty and introduces a multi-agent utility layer. Relative to MARL resource allocation [17, 18], it offers interpretable coordination rather than decentralized learned policies.

3 Problem Formulation

Let the system operate in discrete timesteps $t \in \{1, \dots, T\}$. At each step, the controller observes node telemetry, workload history, predicted demand, uncertainty, and resource inventory,

$$\mathbf{s}_t = \{u_t^{cpu}, u_t^{mem}, w_t, \mathbf{h}_t, q_t, \hat{w}_{t+1}, \sigma_t, r_t\}, \quad (1)$$

where u_t^{cpu} and u_t^{mem} denote selected CPU and memory utilization, w_t is the effective workload at step t , $\mathbf{h}_t$ is the recent workload history used by the predictor, q_t is the resource inventory reported by the resource service, $\hat{w}_{t+1}$ is the one-step-ahead forecast, σ_t is the uncertainty score, and r_t is the normalized risk score.

The action space is

$$\mathcal{A} = \{\text{scale_up}, \text{hold}, \text{scale_down}\}. \quad (2)$$

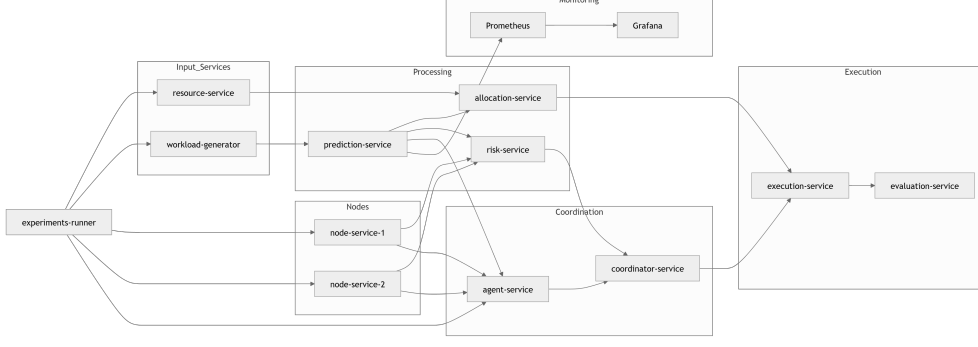


Fig. 1 System architecture of the uncertainty-aware multi-agent scaling framework. The implementation contains eleven FastAPI services, observability components, and a reproducible experiment runner.

The controller must choose $a_t \in \mathcal{A}$ to manage the tension among prediction error, risk exposure, resource cost, and control stability while respecting the currently available node pool. Because the repository evaluates multiple policies over the same predictor, the goal is not to optimize forecasting accuracy alone, but to compare the control consequences of different decision mechanisms under matched experimental conditions.

4 Methodology / Architecture

The framework is organized as independent services connected by HTTP APIs and coordinated by an experiment runner. The **node-service** simulates per-node CPU and memory observations. The **workload-generator** exposes dynamic load signals. The **prediction-service** provides next-step forecasts and uncertainty estimates. The **risk-service** computes a normalized instability score. The **agent-service** emits performance, cost, and risk scores. The **resource-service** tracks a small in-memory node pool, while the **allocation-service** maps requested CPU to a greedy feasible plan. The **execution-service** simulates the chosen scaling action, and the **evaluation-service** records RMSE, utilization, and latency-oriented metrics. Prometheus and Grafana complete the observability path, while the experiment runner handles reset, readiness, execution, logging, and publication exports.

Table 1 summarizes the main services and their roles in the control loop.

At every timestep, the experiment runner collects node telemetry and workload, calls the predictor, computes risk, queries the agent and allocation services, decides an action under the selected policy, executes the action, and records step-level artifacts. This separation is important for two reasons. It mirrors service-oriented deployment settings in which scaling information is naturally distributed, and it makes each control transformation inspectable and individually testable.

Table 1 Service roles in the implemented framework

Service	Role in the pipeline
node-service	Simulates node-local CPU and memory measurements and exposes Prometheus-compatible metrics.
workload-generator	Produces the workload signal consumed by the predictor and experiment runner.
prediction-service	Computes a lightweight recurrent forecast and an uncertainty score from recent history.
risk-service	Converts utilization, uncertainty, trend, volatility, and CPU-memory imbalance into a bounded risk score.
agent-service	Emits performance-oriented, cost-oriented, and risk-oriented scores for the current state.
resource-service	Maintains the in-memory resource inventory for small, medium, and large nodes.
allocation-service	Produces a greedy largest-fitting-first allocation plan for requested CPU demand.
coordinator-service	Implements the central decision API used by the orchestration path.
execution-service	Applies the chosen action and records execution history.
evaluation-service	Tracks forecasting RMSE, utilization statistics, and run-level evaluation metrics.
experiments	Orchestrates repeatable runs, resets services, writes artifacts, aggregates statistics, and exports paper-ready bundles.

5 Mathematical Formulation

5.1 Forecasting and Uncertainty

The prediction service implements a lightweight GRU-style recurrence over a history window $\mathbf{h}_t = [w_{t-m+1}, \dots, w_t]$. For each step in the history, it updates a hidden state using a learned-form recurrence with fixed coefficients,

$$z_i = \sigma(1.4\Delta_i), \quad (3)$$

$$\rho_i = \sigma(-0.8\Delta_i), \quad (4)$$

$$\tilde{h}_i = \tanh(\rho_i h_{i-1} + w_i), \quad (5)$$

$$h_i = (1 - z_i)h_{i-1} + z_i \tilde{h}_i, \quad (6)$$

where $\Delta_i = w_i - w_{i-1}$ and $\sigma(\cdot)$ is the logistic sigmoid. The service then derives the workload level $\ell_t = w_t$, momentum m_t , and volatility v_t from the history and predicts

$$\hat{w}_{t+1} = \text{clip}(\ell_t + m_t + 0.35(h_t - \ell_t), 0, 1), \quad (7)$$

with uncertainty

$$\sigma_t = \min(0.03 + 0.6v_t + 0.25|m_t|, 0.5). \quad (8)$$

These equations are taken directly from the implemented lightweight predictor and make uncertainty increase with both volatility and sharp local trend changes.

5.2 Risk Function

The risk service combines utilization pressure, prediction uncertainty, CPU–memory imbalance, workload trend, and workload volatility into an interpretable additive score. Let

$$\bar{u}_t = \frac{u_t^{cpu} + u_t^{mem}}{2}, \quad b_t = |u_t^{cpu} - u_t^{mem}|. \quad (9)$$

The implemented risk function is

$$r_t = \text{clip}(0.55\bar{u}_t + 0.25\sigma_t + 0.10b_t + 0.20|\tau_t| + 0.20\nu_t, 0, 1), \quad (10)$$

where τ_t is the recent trend signal and ν_t is the recent volatility signal. Equation (10) is monotone in each component before clipping, which makes the score easy to interpret during analysis and ablation.

5.3 Agent Scoring

The multi-agent layer does not learn policies; instead, it decomposes the current state into performance, cost, and risk viewpoints. The implemented scores are

$$s_t^{\text{perf}} = 0.85u_t^{cpu} + 0.15(1 - r_t), \quad (11)$$

$$s_t^{\text{cost}} = 0.70(1 - u_t^{cpu}) + 0.30(1 - r_t), \quad (12)$$

$$s_t^{\text{risk}} = r_t. \quad (13)$$

Equation (11) rewards productive utilization but discounts it when risk is high, whereas Eq. (12) favors low-pressure states. These scores are later combined in the decision logic.

5.4 Objective Formulation

The code base implements a thresholded heuristic controller rather than a directly optimized loss. Nonetheless, the control intent can be written as the minimization of an operational surrogate,

$$\min_{\pi} \sum_{t=1}^T [\lambda_1 |\hat{w}_{t+1} - w_t| + \lambda_2 c_t + \lambda_3 r_t + \lambda_4 \mathbb{I}(a_t \neq a_{t-1})], \quad (14)$$

where c_t denotes a cost proxy and the final term penalizes action churn. Equation (14) reflects the evaluation metrics reported by the publication bundle: forecasting error, cost proxy, risk, and control stability.

Algorithm 1 Uncertainty-Aware Multi-Agent Scaling Policy

Require: workload history $\mathbf{h}_t$
node metrics (u_t^{cpu}, u_t^{mem}) , resource inventory q_t
Ensure: scaling action $a_t \in \{\text{scale_up}, \text{hold}, \text{scale_down}\}$

- 1: Query predictor with $\mathbf{h}_t$ to obtain $(\hat{w}_{t+1}, \sigma_t)$
- 2: Compute trend τ_t and volatility ν_t
- 3: Compute risk r_t using Eq. (10)
- 4: Compute agent scores $s_t^{\text{perf}}, s_t^{\text{cost}}, s_t^{\text{risk}}$ using Eqs. (11)–(13)
- 5: Query resource service for pool q_t
- 6: Query allocation service for plan covering $\hat{w}_{t+1}$
- 7: Compute pressure P_t (Eq. (15))
- 8: Compute release R_t (Eq. (16))
- 9: Set guard $g_t \leftarrow \max(r_t, s_t^{\text{risk}})$
- 10: **if** $P_t \geq 0.58$ **and** $g_t \geq 0.38$ **then**
- 11: $a_t \leftarrow \text{scale_up}$
- 12: **else if** $R_t \geq 0.60$ **and** $r_t \leq 0.42$ **and** $u_t^{cpu} \leq 0.45$ **then**
- 13: $a_t \leftarrow \text{scale_down}$
- 14: **else**
- 15: $a_t \leftarrow \text{hold}$
- 16: **end if**
- 17: Execute a_t
- 18: Record metrics and actions
- 19: Update $\mathbf{h}_{t+1}$ with w_t
- 20: **return** a_t

To operationalize Eq. (14), the proposed family computes a pressure score and a release score,

$$P_t = 0.5\hat{w}_{t+1} + 0.3r_t + 0.2s_t^{\text{perf}}, \quad (15)$$

$$R_t = 0.5(1 - \hat{w}_{t+1}) + 0.3(1 - r_t) + 0.2s_t^{\text{cost}}, \quad (16)$$

and uses the guard

$$g_t = \max(r_t, s_t^{\text{risk}}). \quad (17)$$

The implemented policy is

$$a_t = \begin{cases} \text{scale_up}, & P_t \geq 0.58 \text{ and } g_t \geq 0.38, \\ \text{scale_down}, & R_t \geq 0.60 \text{ and } r_t \leq 0.42 \text{ and } u_t^{cpu} \leq 0.45, \\ \text{hold}, & \text{otherwise.} \end{cases} \quad (18)$$

For comparison, the threshold baseline scales up when $u_t^{cpu} \geq 0.72$ or $w_t \geq 0.78$ and scales down when $u_t^{cpu} \leq 0.38$ and $w_t \leq 0.35$. The reactive baseline uses recent workload change and scales up when $w_t - w_{t-1} \geq 0.12$ or $u_t^{cpu} \geq 0.75$, and scales down when $w_t - w_{t-1} \leq -0.12$ and $u_t^{cpu} \leq 0.45$.

6 Implementation Details

The implementation is written in Python 3.11 and relies on FastAPI plus Uvicorn for service exposure. The microservice decomposition is intentionally fine-grained because the project is both a systems prototype and a research artifact: each stage of the control loop can be queried independently, replaced without collapsing the full stack, and verified with targeted tests. The repository includes unit and integration tests for the risk engine, prediction model, coordinator logic, reporting outputs, and orchestrator behavior.

The allocation layer uses a largest-fitting-first greedy planner over three node types: large (2.0 CPU), medium (0.5 CPU), and small (0.25 CPU). This keeps resource planning distinct from the control policy itself. The execution layer records all chosen actions, which is later used to compute action distribution and action churn. The evaluation layer accumulates RMSE and utilization over the run, while the experiment runner additionally computes average latency, p95/p99 latency, SLO violation rate, throughput proxy, control-loop latency, and cost proxy.

Observability is treated as a first-class feature. Node services export Prometheus-compatible metrics, and Grafana dashboards are provisioned with the repository. Deployment support is provided through Dockerfiles for every service, a top-level `docker-compose.yml`, and Kubernetes manifests under `infra/k8s`. This matters for publication because it makes the artifact portable and makes it possible to regenerate the results under a well-defined environment.

The experiment stack persists reproducibility metadata for every run, including the resolved configuration, config digest, Git status, Python and platform details, workload trace, step logs in CSV and JSONL form, and run summaries. Batch-level reporting exports Markdown, CSV, LaTeX, JSON, and SVG artifacts, which are the direct source of the tables discussed below.

7 Experimental Setup

The primary evidence in this paper comes from the validated publication bundle `results/publication-suite/20260416T145719Z`. This suite contains 390 successful runs out of 390 requested runs, with 30 runs per scenario and 20 timesteps per run. Thirteen scenarios are included: five baseline or extended policies (proposed, threshold, reactive, HPA-like, predictive-only), five ablations (no-risk, no-uncertainty, no-agents, no-trend, no-volatility), one replay-trace scenario, and two normalized Google cluster trace scenarios (proposed and threshold).

The experiments use a deterministic latency configuration with an SLO target of 220 ms, base latency of 35 ms, utilization penalty of 160 ms, queue penalty of 220 ms, and load-change penalty of 90 ms. The risk model uses the weights reported in Eq. (10). The history seed is three timesteps by default, and the runner resets the Docker stack before each run to avoid state contamination between trials.

Metrics were selected to capture both prediction quality and control behavior. Core metrics are RMSE, average utilization, average prediction, and average risk. Extended metrics are average, p95, and p99 latency, p95 utilization, SLO violation rate, control-loop latency, throughput proxy, cost proxy, and action churn. The reporting pipeline



Fig. 2 Experiment workflow from configuration loading to persistent artifact export. Each run resets services, waits for readiness, executes the control loop, and writes both step-level and aggregate evidence.

Table 2 Core policy comparison from the 390-run publication suite

Policy	RMSE	Avg. util.	Avg. risk	Avg. lat. (ms)	Ctrl. lat. (ms)	Cost proxy	Churn	Actions (hold/up/down)
Proposed	0.09	0.6033	0.3820	63.1458	135.5510	0.6908	0.0930	470 / 120 / 10
Threshold	0.09	0.6002	0.3796	63.4635	138.8756	0.6803	0.0000	600 / 0 / 0
Reactive	0.09	0.6015	0.3808	62.9723	120.1461	0.6832	0.1141	565 / 30 / 5
HPA-like	0.09	0.5998	0.3795	63.3113	122.3530	0.6798	0.0000	600 / 0 / 0
Predictive-only	0.09	0.5985	0.3787	63.4467	122.3653	0.6995	0.2632	180 / 360 / 60

also computes permutation p-values, bootstrap confidence intervals for mean differences, Cliff’s delta effect sizes, and Holm-Bonferroni corrected p-values for pairwise comparisons across scenarios. This statistical discipline matters because the project claims only what the generated artifacts support.

8 Evaluation and Results

Table 2 summarizes the main policy comparison used throughout the paper. The most important point is that RMSE is almost invariant across the baseline policies because all of them share the same predictor and the evaluation service computes forecast error against the realized workload independently of actuation. As a result, the informative separation lies in behavior-oriented metrics such as action distribution, cost proxy, and action churn rather than in RMSE alone.

Three observations stand out. First, the threshold policy stays in **hold** for all 600 decisions in the baseline comparison. This is consistent with the implemented thresholds: the measured utilization and workload trajectories rarely cross the aggressive up-thresholds or the simultaneous low-load down-thresholds, so the controller remains static. Second, the predictive-only baseline over-scales. Because it reacts directly to forecast excursions without the risk and agent guards of Eq. (18), it issues 360 scale-up actions and 60 scale-down actions, yielding the highest cost proxy and the highest action churn in the suite. Third, the proposed policy balances adaptivity and restraint. It performs 130 non-hold actions, including both upward and downward adjustments, but keeps action churn far below the predictive-only baseline (0.0930 versus 0.2632).

The reactive baseline is more active than threshold control but still mostly conservative. Its decision rule depends on a relatively large workload-delta threshold (± 0.12), which the synthetic baseline workload does not often exceed. As a result, only 35 of 600 actions are non-hold. The proposed controller therefore differs from reactive control not by making more total actions than any possible competitor, but by using uncertainty-aware pressure and release scores to make bidirectional actions at moments that remain interpretable in the logs.

Table 3 examines how the proposed family changes when specific components are removed. The no-risk ablation collapses the risk score to zero by construction, which

Table 3 Ablation analysis for the proposed controller family

Scenario	Avg. util.	Avg. risk	Avg. lat. (ms)	Ctrl. lat. (ms)	Churn	Actions (hold/up/down)
Proposed	0.6033	0.3820	63.1458	135.5510	0.0930	470 / 120 / 10
No-risk	0.5998	0.0000	63.3513	128.4227	0.0526	533 / 0 / 67
No-uncertainty	0.5987	0.3801	63.6112	139.5424	0.0930	468 / 115 / 17
No-agents	0.6007	0.3796	63.1770	138.8007	0.1193	459 / 134 / 7
No-trend	0.6018	0.3814	62.8980	135.8704	0.0842	471 / 119 / 10
No-volatility	0.6023	0.3811	63.0700	118.1173	0.0912	474 / 112 / 14

Table 4 Trace-driven scenarios from the publication suite

Scenario	Policy	RMSE	Prediction	Avg./P95 lat. (ms)	Ctrl. lat. (ms)	Throughput	Actions (hold/up/down)
Replay trace	Proposed	0.05	0.5685	50.6408 / 67.5560	123.8407	532.5	557 / 41 / 2
Real trace	Proposed	0.07	0.6640	65.2197 / 78.1223	129.4520	627.5	441 / 159 / 0
Real trace	Threshold	0.07	0.6640	64.9013 / 77.7672	150.2136	627.5	600 / 0 / 0

biases the controller toward release decisions and produces 67 scale-down actions with no scale-ups. Removing uncertainty, trend, or volatility does not dramatically change average utilization in this workload, but it does perturb churn and action distribution, which indicates that these signals affect the timing of decisions even when average metrics are similar.

Table 4 addresses generalization beyond the purely generated baseline scenarios. The replay-trace configuration yields lower RMSE and lower latency proxies because its workload trace is smoother and its mean prediction is lower. More importantly, the normalized Google trace scenarios show that the proposed controller remains adaptive on a more realistic workload: it performs 159 scale-up actions, whereas the corresponding threshold policy again remains entirely in `hold`. Although end-to-end latency differences between the two real-trace policies are small, the proposed policy reduces mean control-loop latency from 150.21 ms to 129.45 ms while preserving the same RMSE and throughput proxy.

The pairwise statistical outputs add an important caution. Across the full family of pairwise tests, Holm-Bonferroni correction does not yield rejected null hypotheses at the 0.05 level, even when some raw permutation p-values are small. At the same time, several behavioral comparisons exhibit large effect sizes. For example, the proposed-versus-threshold comparison shows a large effect on action churn (Cliff’s $\delta = 0.7$) despite negligible differences in average utilization and latency proxies, and predictive-only versus proposed shows a large effect on both action churn ($\delta = 1.0$) and control-loop latency ($\delta = -0.74$). Accordingly, the strongest defensible claim is not that the proposed controller strictly dominates all baselines on every metric, but that it induces qualitatively different and more balanced control behavior under matched conditions.

9 Discussion

The empirical picture that emerges is subtle but useful. The proposed controller does not outperform the baselines by a wide margin on mean RMSE or average utilization, and that outcome is understandable because the predictor is shared and the workload regime is relatively controlled. What changes is the structure of the decision process. Threshold and HPA-like baselines behave like static guards; predictive-only control

is eager but insufficiently tempered; the proposed controller inserts uncertainty and agent-derived signals between prediction and action, which reduces the tendency to overreact while still permitting upward and downward motion.

This positioning aligns with the literature. Unlike MagicScaler [6], the framework does not rely on Gaussian-process forecasting or stochastic optimization, so it is less expressive in uncertainty modeling but dramatically lighter to run and easier to reproduce. Unlike RL-based autoscalers such as the DQN microservice controller of Nayir *et al.*, MSARS, or workflow SARSA/Q-learning controllers [11, 12, 15], it avoids policy-training overhead and opaque reward shaping. That simplicity is valuable in a research artifact intended for repeatable experimentation. At the same time, it means the framework cannot yet exploit long-horizon reward structure in the way RL can.

Compared with Kubernetes HPA-style baselines [14], the proposed system introduces two additional layers of reasoning: explicit uncertainty-aware risk and multi-agent utility decomposition. The real-trace experiments are especially relevant here. Both real-trace controllers reach identical RMSE, prediction, and throughput proxies, but only the proposed controller adapts through 159 scale-up actions and a materially lower control-loop latency. This suggests that the framework captures some of the responsiveness sought in cloud-native controllers without degenerating into the over-scaling pattern of the predictive-only baseline.

Compared with MARL resource allocation [17, 18], the present work occupies a middle ground. It adopts the idea that resource management is multi-objective and naturally decomposed across perspectives, but it stops short of decentralized learned coordination. That is a limitation for optimality, yet a strength for auditability. Every intermediate quantity in Eqs. (10)–(18) can be logged, inspected, and perturbed by ablation, which is precisely what the publication bundle demonstrates.

A final contribution of the framework is methodological rather than algorithmic. The literature repeatedly notes the difficulty of comparing autoscaling methods across incompatible traces, metrics, and reporting conventions [4, 19]. By packaging per-run provenance, workload traces, aggregate summaries, pairwise tests, and paper-ready tables, the repository addresses reproducibility more directly than many prototype papers do.

10 Limitations

The study has three primary limitations. First, much of the headline evidence still comes from synthetic generated workloads and deterministic latency proxies. The replay-trace and normalized Google trace scenarios improve realism, but they do not substitute for a live deployment on a real Kubernetes cluster with production traffic. Second, the predictor, risk model, and multi-agent layer are intentionally lightweight. This keeps the system fast and interpretable, yet it also means the controller does not represent the strongest forecasting, uncertainty quantification, or learning methods available in the literature. Third, the empirical separations are strongest in behavior-oriented metrics rather than in direct QoS or cost measurements from an operational system. The project should therefore be interpreted as a strong research prototype rather than a finalized production autoscaler.

A further limitation is statistical. The reporting pipeline is rigorous and includes corrected multiple comparisons, but the full Holm-Bonferroni correction across the scenario family is conservative enough that no pairwise comparison is rejected at the 0.05 level. This reinforces the need for cautious interpretation and motivates larger-scale or more heterogeneous evaluations in future work.

11 Future Work

Several extensions are natural. The most obvious is to replace the heuristic controller of Eq. (18) with an RL-based scaling policy that can optimize Eq. (14) directly, enabling comparisons against DQN, TD3, SARSA, or hierarchical RL in the same experimental harness [7, 11, 12, 15]. A second direction is stronger probabilistic forecasting: conformal or Bayesian predictors could replace the current lightweight recurrent model and make the uncertainty term more faithful to the uncertainty-aware literature represented by MagicScaler and Aquatope [6, 13]. A third direction is explicit SLO-aware and graph-aware scaling, in which inter-service dependencies and latency budgets are first-class policy inputs. Finally, the artifact should be exercised on larger and more diverse real trace datasets and, ideally, a real deployment, so that the current claims about adaptivity and reproducibility can be tested under richer operational conditions.

12 Conclusion

This paper presented a complete publication-grade description of a multi-agent framework for uncertainty-aware resource allocation and auto-scaling in distributed systems. The work integrates implementation, validated experiments, and curated literature rather than treating them separately. The resulting framework combines a lightweight recurrent predictor, an interpretable risk score, multi-agent utility decomposition, greedy allocation, centralized coordination, and a reproducible experiment pipeline. Empirically, the proposed controller does not separate strongly from the baselines on mean RMSE or utilization in the synthetic baseline regime, but it does produce a more balanced action profile than threshold and predictive-only baselines and preserves adaptive behavior under trace-driven evaluation. Methodologically, the stronger contribution is the reproducible artifact: 390 successful runs, exported statistical comparisons, and manuscript-ready outputs that make the system inspectable and extensible. For systems researchers, that combination of architectural completeness, uncertainty-aware control, and reproducibility is a useful foundation for subsequent work on learning-based, SLO-aware, and deployment-backed autoscaling.

13 Declarations

Funding. No external funding was used for the work reported in this manuscript.

Conflict of interest. The author declares no conflict of interest.

Availability of data and materials. The results reported in this paper are derived from the project repository and the validated artifact bundle

results/publication-suite/20260416T145719Z. The repository also contains the replay trace sample and the normalized Google cluster extract used for trace-driven evaluation.

Code availability. Source code, experiment configurations, reporting scripts, and the reproducibility package generator are included in the project repository.

Ethics approval. Not applicable.

Consent to participate. Not applicable.

Consent for publication. Not applicable.

References

- [1] I. Pintye, J. Kovacs, and R. Lovas, “Enhancing Machine Learning-Based Autoscaling for Cloud Resource Orchestration,” *Journal of Grid Computing*, vol. 22, Art. 68, 2024. doi: 10.1007/s10723-024-09783-1.
- [2] M. Catillo, U. Villano, and M. Rak, “A survey on auto-scaling: how to exploit cloud elasticity,” *International Journal of Grid and Utility Computing*, vol. 14, no. 1, pp. 37–50, 2023. doi: 10.1504/IJGUC.2023.129702.
- [3] B. Jeong and Y.-S. Jeong, “Autoscaling techniques in cloud-native computing: A comprehensive survey,” *Computer Science Review*, vol. 58, Art. 100791, 2025. doi: 10.1016/j.cosrev.2025.100791.
- [4] D. B. Bodra and S. K. Khairnar, “Machine learning-based cloud resource allocation algorithms: a comprehensive comparative review,” *Frontiers in Computer Science*, vol. 7, Art. 1678976, 2025. doi: 10.3389/fcomp.2025.1678976.
- [5] R. Ghafari and N. Mansouri, “Reinforcement learning-based solution for resource management in fog computing: A comprehensive survey,” *Expert Systems with Applications*, vol. 276, Art. 127214, 2025. doi: 10.1016/j.eswa.2025.127214.
- [6] Z. Pan *et al.*, “MagicScaler: Uncertainty-aware, Predictive Autoscaling,” *Proceedings of the VLDB Endowment*, vol. 16, no. 12, pp. 3808–3821, 2023. doi: 10.14778/3611540.3611566.
- [7] L. Chen, C. Jiang, Y. Huang, and W. Wu, “Uncertainty-aware cloud resource management via fuzzy-adaptive hierarchical reinforcement learning,” *Applied Soft Computing*, vol. 186, Art. 114157, 2026. doi: 10.1016/j.asoc.2025.114157.
- [8] M. Doostmohammadian *et al.*, “Survey of distributed algorithms for resource allocation over multi-agent systems,” *Annual Reviews in Control*, vol. 52, pp. 100983:1–33, 2025. doi: 10.1016/j.arcontrol.2024.100983.
- [9] Y. Liu, Z. Shi, J. Sun, and X. Wang, “Multi-agent reinforcement learning for resources allocation optimization: a survey,” *Artificial Intelligence Review*, 2025. doi: 10.1007/s10462-025-11340-5.

- [10] K. Senjab *et al.*, “A survey of Kubernetes scheduling algorithms,” *Journal of Cloud Computing*, vol. 12, Art. 87, 2023. doi: 10.1186/s13677-023-00471-1.
- [11] H. Nayir, A. Demir, and H. Kaya, “Multi-Dimensional Autoscaling in Microservices Using Reinforcement Learning,” in *Proceedings of the IEEE/ACM International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*, 2025, pp. 1–8. doi: 10.1109/MASCOTS67699.2025.11283269.
- [12] K. Hu, L. Wen, M. Xu, and K. Ye, “MSARS: A Meta-Learning and RL Framework for SLO Resource Allocation and Adaptive Scaling in Microservices,” arXiv:2409.14953, 2024.
- [13] Z. Zhou, Y. Zhang, and C. Delimitrou, “QoS-Aware Resource Management for Multi-phase Serverless Workflows with Aquatope,” arXiv:2212.13882, 2022.
- [14] F. H. Buzato and A. Goldman, “Microservices Performance Optimization Through Horizontal Pod Autoscaling: A Comprehensive Study,” *International Journal of Networking and Computing*, vol. 16, no. 1, pp. 52–75, 2026. doi: 10.15803/ijnc.16.1_52.
- [15] Y. Gari, E. Pacini, L. Robino, C. Mateos, and D. A. Monge, “Online RL-based cloud autoscaling for scientific workflows: Q-Learning vs. SARSA,” *Future Generation Computer Systems*, vol. 157, pp. 573–586, 2024. doi: 10.1016/j.future.2024.04.014.
- [16] L. Xu, J. Tan, and H. Zhuang, “Enhanced multi-agent DRL for task offloading and resource allocation in vehicular networks,” *Digital Communications and Networks*, vol. 12, no. 1, pp. 66–75, 2026. doi: 10.1016/j.dcan.2025.09.002.
- [17] G. Yao, H. Liu, and L. Dai, “Multi-Agent RL for Adaptive Resource Orchestration in Cloud-Native Clusters,” arXiv:2508.10253, 2025.
- [18] K. Mao, “Multi-agent RL-driven resource game optimization for network slicing in MEC-enabled HetNets,” *Scientific Reports*, vol. 16, Art. 3231, 2026. doi: 10.1038/s41598-025-33190-5.
- [19] V. S. Machiraju, V. Kumar, and S. Sharma, “ML-Based Autoscaling for Elastic Cloud Applications: Taxonomy, Frameworks, and Evaluation,” *Mathematical and Computational Applications*, vol. 31, no. 2, Art. 49, 2026. doi: 10.3390/mca31020049.